

Internettechnologien 2 – Andreas Baumgart

Dokumentation

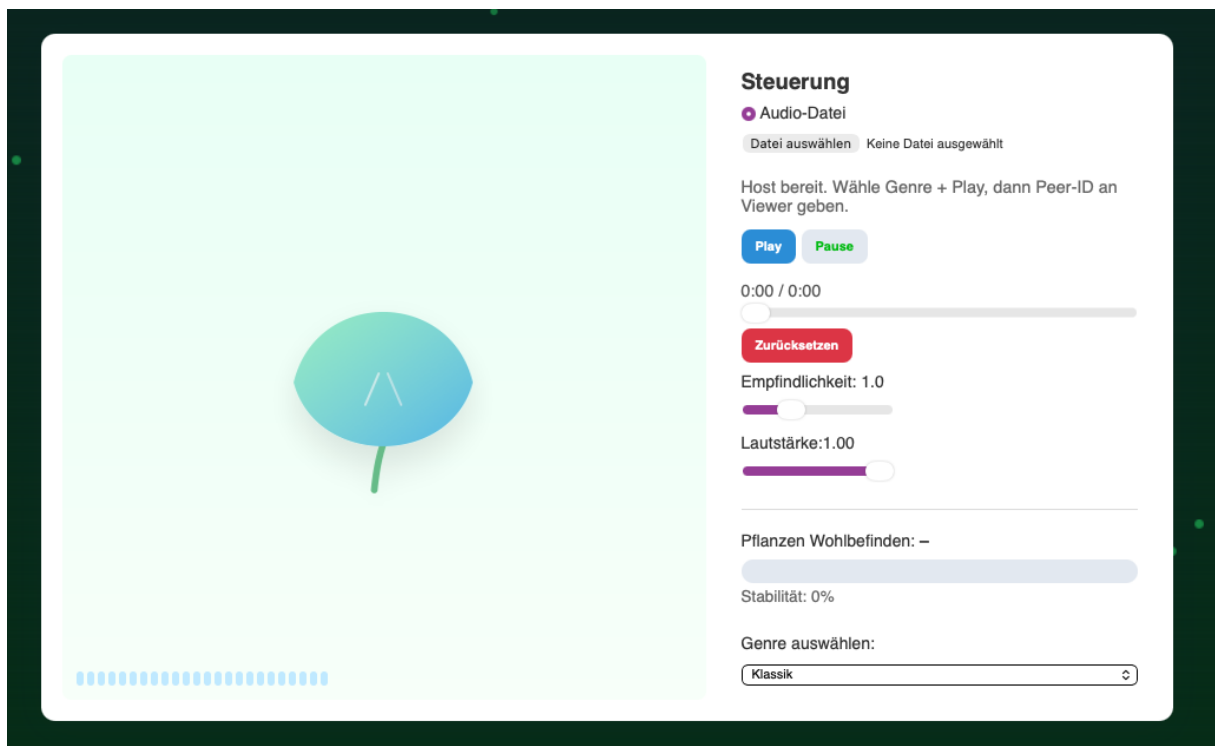
„Synth Garden“

Gruppe 1

Katarzyna Wisniewska (6060784) katarzyna.wisniewska@student.jade-hs.de

Malina Krüger (6047560) malina.krueger@student.jade-hs.de

Nina Michalski (6050629) nina.michalski@student.jade-hs.de



Diese Website läuft fehlerfrei auf folgenden Browsern: Safari und Google Chrome

Malina Krüger (MK)

Nina Michalski (NM)

Katarzyna Wisniewska (KW)

1. Wie hast du das verwendete Framework eingesetzt?

Das Framework p5.js wurde verwendet, um die grafischen Darstellungen und Animationen umzusetzen. Mit der Erweiterung p5.sound konnte zusätzlich Audio aufgenommen und analysiert werden. Zum Beispiel konnte mit Hilfe der Funktion `fft.getEnergy("bass")`, die Bassstärke des Audios ausgelesen werden. Anhand des Ergebnisses wurde das Blatt visuell verändert, sodass es sich stärker bewegt, wenn viele tiefe Töne gespielt werden.

2. Wie gut haben die Online-Tutorials geholfen, ein eigenes Projekt umzusetzen?

MK: Im Rahmen dieser Kursarbeit bestand das Ziel darin, eine interaktive Web-Anwendung zu entwickeln, die Audiosignale aus musikalischen Quellen visuell darstellt und diese audiovisuelle Kombination, als Stream für andere Nutzer verfügbar macht.

Ausgehend von dieser Aufgabenstellung haben wir ein Projekt entwickelt, das Audiosignale in organische, pflanzenähnliche Visuals übersetzt. Die Grundidee war es, Musik nicht nur technisch zu analysieren, sondern sie visuell erfahrbar zu machen. Statt einer klassischen Darstellung wie einem Oszilloskop oder einem Spektrogramm sollte eine metaphorische Visualisierung entstehen, bei der Klangparameter das Wachstum, die Bewegung und den Zustand einer digitalen Pflanze beeinflussen. Dadurch sollte eine emotionale und ästhetische Verbindung zwischen Sound und Visualisierung geschaffen werden.

NM: Die verfügbaren Online-Tutorials zur Web Audio API und zu p5.js waren eine sehr gute Grundlage für den Einstieg in die Thematik, dienten aber vor Allem als theoretische Basis. Sie vermittelten das notwendige Verständnis für den technischen Aufbau des AudioContexts, die Erstellung von AnalyseNodes sowie das grundlegende Auslesen von Frequenzdaten. Ohne das Basiswissen wäre das Zugreifen auf die Audiodaten nicht möglich gewesen. Allerdings sind wir bei der konkreten Projektumsetzung an Grenzen gestoßen, da die meisten Tutorials nur isolierte Funktionen behandelt haben, wie etwa ein einfaches Oszilloskop. Die spezifische Anforderung unserer Kursarbeit, die eine

Kombination aus interaktiver Genre-Visualisierung aufzeigt, komplexer Steuerung (Pause/Resume/Spulen) und der Vorbereitung auf Streaming, wurde in keinem Tutorial komplett behandelt. Daher mussten wir die Konzepte aus verschiedenen Quellen herausnehmen und eigenständig zu einer neuen, komplexeren Gesamtheit zusammenfügen.

MK: Für die Umsetzung der Visuals haben wir uns für das JavaScript-Framework p5.js entschieden. Ausschlaggebend für diese Wahl war der kreative Fokus des Frameworks sowie die einfache Möglichkeit, dynamische Grafiken und Animationen zu erstellen. p5.js eignete sich besonders gut, um organische Formen, Partikelsysteme und fließende Bewegungen zu realisieren. Im Projekt wurde p5.js eingesetzt, um Blattformen zu zeichnen, Partikel zu animieren und visuelle Parameter direkt an Audiodaten zu koppeln. Die Visuals reagieren dabei in Echtzeit auf Veränderungen im Klang, insbesondere auf den Bassanteil des Audiosignals.

3. Welche Beispiele musstest du selbst entwerfen, um dich einzuarbeiten?

NM: Um die Anforderungen an Interaktivität und Ästhetik zu erfüllen, mussten wir mehrere Komponenten selbst entwerfen, die über Standard-Beispiele hinausgehen. Ein zentrales Element war das Modell für das Genre-Mapping. Wir haben ein System entwickelt, das abstrakte Audio-Parameter in spezifischen visuellen Eigenschaften abbildet, sodass zum Beispiel „Jazz“ andere Farben und Bewegungsdynamiken auslöst als „Techno“. Außerdem haben wir den Aufbau für den Fortschrittsregler komplett selbst entwickelt. Die Standard-Browser-Events haben wir so synchronisiert, dass die Animationsschleife und die manuelle Benutzereingabe beim Spulen nicht gegeneinander arbeiten und sich überschneiden. Auch die Logik für eine stabile Pause- und Resume-Funktion, die visuelle Artefakte beim Fortsetzen der Musik vermeidet, hat eine eigenständige Entwicklung jenseits vorgefertigter Code-Snippets angefordert.

MK: Zur Einarbeitung in p5.js und in das Thema Audio-Visualisierung habe ich verschiedene Online-Tutorials und Beispielprojekte bearbeitet. Diese Tutorials waren vor allem hilfreich, um grundlegende Konzepte zu verstehen, etwa den Umgang mit dem Canvas, die Struktur eines p5-Sketches. Allerdings haben mir viele dieser Tutorials nur begrenzt bei der konkreten Umsetzung unseres eigenen Projekts geholfen. Insbesondere bei der Kombination von Web Audio API, Visualisierung und Streaming waren viele Beispiele zu allgemein oder auf andere Anwendungsfälle ausgelegt. In diesen Situationen reichten Tutorials allein nicht aus, um die auftretenden Probleme zu lösen.

Um uns die notwendigen Kenntnisse anzueignen, haben wir daher eigene kleine Experimente und Prototypen entwickelt. Dazu gehörten einfache Audio-Balkenvisualisierungen, Tests zur Bass-Erkennung sowie isolierte Versuche mit WebRTC ohne visuelle Komponenten. Diese selbst entworfenen Beispiele waren entscheidend, um ein tieferes Verständnis für die Zusammenhänge zwischen Audioanalyse, Visualisierung und Streaming zu entwickeln. Erst durch dieses experimentelle Vorgehen konnten wir die einzelnen technischen Bausteine sinnvoll miteinander verbinden.

4. Wie hast du dich inhaltlich organisiert, um das Thema zu bearbeiten?

KW: Um das Thema erfolgreich zu bearbeiten, haben wir uns zuerst gemeinsam die Aufgabenstellung angeschaut und Ideen gesammelt. Danach haben wir gemeinsam überlegt, wie wir unsere Ideen umsetzen können und welche Aufgaben erledigt werden müssen. Die Aufgaben wurden immer innerhalb unserer Gruppe verteilt, sodass jedes Gruppenmitglied für bestimmte Inhalte verantwortlich war. Durch regelmäßige Absprachen in den Vorlesungen oder auch Zoom Meetings konnten wir sicherstellen, dass alle Inhalte gut zusammenpassen und mögliche Fehler gefunden und behoben werden.

MK: Inhaltlich haben wir unsere Arbeit in mehrere Phasen unterteilt. Zunächst lag der Fokus auf der lokalen Audioanalyse und der Visualisierung ohne Streaming. Anschließend wurde das visuelle Konzept weiterentwickelt und verfeinert. Erst danach haben wir uns intensiv mit dem Streaming-Aspekt beschäftigt. Rückblickend müssen wir feststellen, dass wir die Komplexität von WebRTC deutlich unterschätzt haben. Zu Beginn sind wir davon ausgegangen, dass sich Streaming relativ schnell

integrieren lässt. Tatsächlich erwies sich dieser Teil des Projekts als der zeitintensivste und anspruchsvollste. Ein großer Teil der Arbeitszeit floss in Fehlersuche, Debugging und das Umstrukturieren des Codes. Unsere ursprüngliche Zeitplanung war hier zu optimistisch, was wir im Nachhinein kritisch reflektieren.

5. Wie hast du die JavaScript-Audio API genutzt?

KW: Wir haben die JavaScript-Audio API genutzt, um Audiodateien im Browser abzuspielen und zu analysieren. Dazu wurde ein Audio-Element mit einem Audio-Context verbunden. Mithilfe von einem Analyser-Node wurden Frequenzdaten und Lautstärkedaten des Audiosignals in Echtzeit gemessen, um grafische Elemente wie das SVG-Blatt und visuelle Balken dynamisch an die Musik anzupassen. Zusätzlich haben wir Funktionen zur Steuerung der Wiedergabe, Lautstärke und Empfindlichkeit eingebaut.

MK: Für die Audioverarbeitung haben wir die Web Audio API eingesetzt. Sie diente dazu, Audiosignale zu analysieren und Frequenzdaten auszulesen. Mithilfe eines AnalyserNodes wurden FFT-Daten gewonnen, aus denen Parameter wie Bass-, Mittel- und Höhenanteile berechnet wurden. Diese Werte bildeten die Grundlage für die visuelle Reaktion der Anwendung. Bewegungen, Skalierungen und Farbveränderungen der Visuals wurden direkt an diese Audio-Parameter gekoppelt, sodass Klangveränderungen unmittelbar sichtbar wurden.

6. Welche Prototypen-Idee hast du entwickelt? Wie gut konntest du sie umsetzen?

Unsere Prototypen-Idee war es eine interaktive Webseite, auf der der Nutzer durch Musik Pflanzen in Echtzeit verändern kann. Dabei sollten verschieden Musik Genres und Audioeigenschaften die Merkmale, wie Farbe, Form oder Größe, der Pflanzen beeinflussen. Unsere Idee konnten wir größtenteils umsetzen, da Audiodateien abgespielt werden, analysiert werden und die visuellen Elemente sich an die Musik anpassen. Einige Details konnten wir leider nur vereinfacht umsetzen, das Grundkonzept der interaktiven Pflanzen wurde jedoch erfolgreich umgesetzt.

7. Welche Probleme gab es, und wie hast du sie gelöst?

NM: Bei der Entwicklung sind verschiedene technische Probleme aufgetreten, die wir schrittweise lösen mussten. Ein Hindernis, das schon zu Beginn aufgetreten ist, war die Inkonsistenz bei Dateipfaden, da der Browser die MP3-Dateien aufgrund von Groß- und Kleinschreibung im Ordernamen „Musik“ versus „musik“ nicht finden konnte und Fehlermeldungen anzeigte. Wir haben es gelöst durch eine strikte einheitliche Kleinschreibung aller Pfade und Dateinamen. Ein weiteres Problem hat sich beim Zurücksetzen des Players gezeigt: Die Farbe des Blattes passte sich nach einem Neustart oft nicht korrekt an das ausgewählte Genre an. Unsere Lösung war, den Funktionsaufruf für die Farbgebung direkt in der Phase nach dem Reset neu zu platzieren. Außerdem gab es einen Konflikt bei der Steuerung des Fortschrittsbalkens, da die automatische Aktualisierung durch die Animationsschleife, die manuelle Eingabe des Nutzers überschrieben hat. Dieses Problem behoben wir durch die Einführung einer Status-Variable, die erkennt, ob der Nutzer gerade den Regler zieht, und in dem Fall das automatische Update kurzzeitig pausiert.

MK: Während der Entwicklung des Projekts traten mehrere technische und konzeptionelle Probleme auf, die den Arbeitsprozess maßgeblich beeinflussten. Ein zentrales Problem bestand zunächst in der Komplexität der Anwendung selbst: Audioanalyse, visuelle Darstellung und Streaming mussten gleichzeitig funktionieren und korrekt miteinander verbunden sein. Fehler ließen sich daher nicht immer eindeutig einer einzelnen Komponente zuordnen, was die Fehlersuche erschwerte.

Ein wiederkehrendes Problem war, dass Verbindungen zwischen Host und Viewer zwar scheinbar erfolgreich aufgebaut wurden, jedoch kein Audio oder keine auswertbaren Daten auf der Empfängerseite ankamen. Besonders irritierend war dabei, dass Statusmeldungen und Peer-Verbindungen korrekt wirkten, während die eigentliche Funktionalität fehlte. Die Ursache lag häufig in falsch verknüpften Audio-Nodes, nicht weitergereichten MediaStreams oder in Zuständen, in denen Audio zwar abgespielt, aber nicht analysiert wurde. Gelöst wurde dieses Problem durch eine konsequente Trennung der einzelnen Verarbeitungsschritte sowie durch das schrittweise Testen jeder Komponente für sich, bevor sie wieder zusammengeführt wurde.

Ein weiteres Problem betraf die Synchronisation von Audio und Visualisierung. In frühen Versionen reagierten die Visuals entweder verzögert oder gar nicht auf den Sound. Ursache war hier meist, dass Analyse- und Rendering-Schleifen nicht sauber initialisiert oder mehrfach gestartet wurden. Durch eine klar strukturierte Initialisierung des AudioContext, die Verwendung eines zentralen AnalyserNode und eine kontrollierte Animationsschleife konnte dieses Verhalten stabilisiert werden.

Auch Entwicklungswerkzeuge stellten eine Herausforderung dar. Fehlerhinweise aus Browsern oder Editoren wie Dreamweaver waren teilweise irreführend oder bezogen sich auf Syntaxprobleme, die zur Laufzeit keine Rolle spielten. Dies führte zunächst zu Verunsicherung, bis klar wurde, dass funktionierende Laufzeitlogik wichtiger ist als statische Warnungen. Die Lösung bestand darin, sich stärker auf die Browserkonsole und das tatsächliche Laufzeitverhalten zu konzentrieren.

Nicht zuletzt stellte die Testumgebung selbst ein Problem dar. Das parallele Testen von Host und Viewer in mehreren Browser-Tabs, Fenstern oder auf unterschiedlichen Geräten führte zu unerwarteten Seiteneffekten, insbesondere bei lokalem Hosting. Diese Probleme konnten durch klar definierte Testabläufe, getrennte Fenster und gezielte Tests mit lokalen Netzwerkadressen reduziert werden.

8. Welche Rolle spielte ChatGPT oder andere KI-Tools bei deiner Arbeit?

NM: ChatGPT war in unserem Arbeitsprozess ein effizienter Assistent für Debugging und Syntax-Fragen, aber hat nicht die konzeptionelle Arbeit übernommen. Das Tool hat uns dabei geholfen, Fehler in der Nutzung der Web Audio API schneller zu identifizieren und hat nützliche Boiler Plate-Codes, etwa für die Formatierung von Zeitangaben oder die Grundstruktur von SVG-Elementen geliefert. Dies steigerte unsere Effizienz sehr, da wir weniger Zeit mit der Suche nach Syntax-Details verbracht haben und uns stärker auf die Integrationslogik konzentrieren konnten. Die kreativen Entscheidungen, wie das Design der Visuals, die Definition von den Genre-Parametern und die Wahl der Streaming-Architektur, wurden jedoch eigenständig vom Team getroffen.

MK: Eine zentrale Rolle bei der Bewältigung dieser Herausforderungen spielte der Einsatz von KI-Tools, wie ChatGPT oder Perplexity. Während Tutorials oft keine konkreten

Antworten auf unsere individuellen Probleme lieferten, konnte die KI gezielt beim Analysieren von Code, beim Finden von Fehlern und beim Entwickeln alternativer Lösungsansätze helfen. Die KI wurde dabei nicht als Ersatz für eigenes Denken genutzt, sondern als unterstützendes Werkzeug, um komplexe Zusammenhänge besser zu verstehen und reflektiert umzusetzen.

9. Wie funktionieren analoge Synthesizer grundsätzlich?

Analoge Synthesizer basieren auf Erzeugung und Formung von Klängen durch elektrische Spannungen. Das Grundprinzip ist ein modularer Signalfluss, der meistens bei einem spannungsgesteuerten Oszillator (VCO) beginnt. Dieser erzeugt die grundlegende Schwingungsform, wie zum Beispiel eine Sägezahn- oder Rechteckwelle, und bestimmt die Tonhöhe. Das Signal läuft anschließend durch einen spannungsgesteuerten Filter (VCF), der das Klangspektrum bearbeitet, indem er bestimmte Frequenzen abschneidet oder betont, was ausschlaggebend für den Klangcharakter ist. Die Lautstärke und dessen Verlauf werden schließlich durch einen spannungsgesteuerten Verstärker (VCA) und Hüllkurvengeneratoren (ADSR) geformt. Zusätzlich können Modulationsquellen wie Low-Frequency Oscillators (LFO) eingesetzt werden, um Parameter wie Tonhöhe oder Filterfrequenz rhythmisch zu verändern.

10. Streaming-spezifisch:

10.1. Welche Variante hast du gewählt (A oder B)? Warum?

Für das Streaming-Konzept haben wir uns bewusst für Variante B entschieden, bei der nur das Audiosignal gestreamt wird und die Visuals bei den Zuschauern lokal berechnet werden. Diese Entscheidung haben wir getroffen, weil diese Variante ressourcenschonender ist und sich besser skalieren lässt. Zudem erlaubt sie eine saubere Trennung zwischen Audioübertragung und visueller Darstellung. Gerade für audio-reaktive Visualisierungen ist es sinnvoll, die Berechnung lokal durchzuführen, da so Verzögerungen reduziert und Performance-Probleme vermieden werden können.

10.2. Wie hast du **Audio + Video synchronisiert**?

Die Synchronität zwischen Audio und Visuals ergibt sich dadurch, dass beide auf denselben Audio-Stream reagieren.

10.3. Wie hast du das **Streaming technisch umgesetzt (WebRTC, Server, Peer-to-Peer)?**

Die technische Umsetzung des Streamings erfolgte mithilfe von WebRTC in einer Peer-to-Peer-Struktur. Über einen PeerJS-Server wurde die Verbindung zwischen Host und Viewer hergestellt. Der Host startet den Stream und erhält eine Peer-ID, die von den Zuschauern eingegeben wird, um sich mit dem Stream zu verbinden. Das Audiosignal wird anschließend direkt zwischen den Peers übertragen, während die Visualisierung auf Basis des empfangenen Audios lokal berechnet wird.

10.4. Wie bist du mit **Latenz und Synchronität umgegangen?**

Die Synchronität zwischen Audio und Visuals ergibt sich dadurch, dass beide auf denselben Audio-Stream reagieren.

10.5. Welche Einstellungen musstest du beim Audio-Input beachten (z. B. Echo-Cancellation aus)?

Beim Mikrofon-Input haben wir Echo-Cancellation, Noise-Suppression und Auto-Gain deaktiviert und Sample-Rate/ChannelCount gesetzt.

10.6. Wie hast du die **Visualisierung dynamisch an Parameteränderungen angepasst?**

Die Visualisierung wurde dynamisch an Parameteränderungen angepasst, indem wir kontinuierlich Frequenzdaten aus der Web Audio API ausgelesen und ausgewertet haben. Mithilfe eines AnalyserNodes haben wir pro Frame das aktuelle Frequenzspektrum des Audiosignals erfasst und daraus verschiedene Kenngrößen berechnet, darunter den Bass-, Mitten- und Höhenanteil sowie einen durchschnittlichen Energie-Wert. Diese Werte wurden anschließend normalisiert und gezielt auf visuelle Eigenschaften gemappt.

Der Bassanteil steuert vor allem Bewegung, Rotation und Skalierung der zentralen Blattformen, sodass tieffrequente Impulse deutlich sichtbar werden. Mitten- und Hochfrequenzen beeinflussen unter anderem die Intensität der Balkenvisualisierung und die allgemeine Dynamik der Animation. Zusätzlich werden Farbveränderungen und Amplituden der Balken direkt aus den Frequenzwerten abgeleitet. Durch dieses Mapping reagieren alle visuellen Elemente in Echtzeit auf Veränderungen im Klangbild. Auf diese Weise entsteht eine direkte und nachvollziehbare Verbindung zwischen Audio-Parametern und visueller Darstellung, bei der musikalische Dynamik und Charakter unmittelbar sichtbar werden.

10.7. Wie hast du die Streaming-Lösung getestet (z. B. mehrere Zuschauer gleichzeitig)?

Die Streaming-Lösung haben wir schrittweise getestet, um Fehlerquellen klar identifizieren zu können. Zunächst erfolgten Tests lokal in mehreren Browser-Tabs auf demselben Rechner, um grundlegende Verbindungsprobleme auszuschließen. Anschließend haben wir verschiedene Geräte im gleichen Netzwerk eingesetzt, etwa einen zweiten Laptop oder ein Smartphone, um reale Nutzungsszenarien zu simulieren. Dabei wurde überprüft, ob mehrere Zuschauer gleichzeitig eine Verbindung herstellen können und ob der Audio-Stream stabil bleibt. Zusätzlich haben wir wiederholt das Verbinden und Trennen von Zuschauern getestet, um sicherzustellen, dass neue Verbindungen korrekt aufgebaut werden und bestehende Streams nicht abbrechen. Diese Tests waren essenziell, da viele Probleme erst unter realistischen Bedingungen sichtbar wurden, etwa Verzögerungen, fehlende Audioübertragung oder fehlerhafte Statusanzeigen. Durch dieses iterative Testen konnten wir die Streaming-Funktion Schritt für Schritt stabilisieren.

11. Was waren deine größten Herausforderungen, und wie hast du sie gelöst?

NM: Eine der größten technischen Herausforderungen war die Synchronisation der HTML-Benutzeroberfläche mit der Audio-Verarbeitung. Da die Animationsschleife sechzigmal

pro Sekunde läuft, hat das zu Konflikten mit der manuellen Steuerung von dem Spul-Regler geführt. Wir haben das durch eine komplexe Event-Steuerung mittels Boolescher Variablen gelöst, die zwischen automatischem Update und Benutzerinteraktion unterscheidet. Eine weitere, eher konzeptionelle Herausforderung ist die lokale Dateiwiedergabe bis hin zum Echtzeit-Streaming. Die Umstellung der Audioquelle von statischen Dateien auf einen Live-Input via WebRTC brauchte ein tiefes Verständnis von Netzwerk-Streams. Wir haben uns dieser Herausforderung gestellt, indem wir uns für eine Architektur entschieden haben, bei der nur das Audiosignal gestreamt wird und die Visualisierung lokal beim Empfänger berechnet wird, um Latenzprobleme zu minimieren.

MK: Die größten Herausforderungen lagen eindeutig im Bereich des Streamings und der zeitlichen Einschätzung des Arbeitsaufwands. Besonders die Arbeit mit WebRTC stellte sich als deutlich komplexer heraus, als wir zu Beginn angenommen hatten. Obwohl grundlegende Beispiele und Dokumentationen vorhanden sind, erwies sich die praktische Umsetzung als schwierig, da viele Probleme erst im Zusammenspiel von Audio, Browser, Peer-Verbindungen und lokalen Testumgebungen sichtbar wurden. Fehlermeldungen waren häufig unspezifisch oder traten nur unter bestimmten Bedingungen auf, etwa beim Wechsel zwischen Tabs, Geräten oder Netzwerken.

Die Fehlersuche in diesem Bereich war entsprechend mühsam und teilweise frustrierend. Oft funktionierte ein Teil der Anwendung korrekt, während ein anderer scheinbar grundlos versagte. Gerade bei WebRTC war es schwierig zu unterscheiden, ob ein Problem durch die Peer-Verbindung, den Audio-Stream, den Browser oder die eigene Logik verursacht wurde. Gleichzeitig führte die intensive Auseinandersetzung mit dem Thema dazu, dass wir uns zeitweise sehr stark darin vertieft haben. Sobald ein konkretes Problem identifiziert war, gerieten wir regelrecht in eine „Arbeitsblase“, in der wir über längere Zeit ausprobierten, testeten und an Details feilten, teilweise ohne das Vergehen der Zeit bewusst wahrzunehmen.

Rückblickend war die größte Herausforderung weniger ein einzelnes technisches Problem, sondern vor allem die Zeitplanung. Der Aufwand für das Streaming – insbesondere für WebRTC – wurde von uns deutlich unterschätzt. Während Audioanalyse und Visualisierung relativ schnell greifbare Ergebnisse lieferten, beanspruchte das Debugging der Streaming-Funktion einen unverhältnismäßig großen Teil der verfügbaren

Zeit. Gelöst haben wir diese Herausforderung letztlich durch eine schrittweise Vereinfachung der Anwendung, klare Trennung von Host- und Viewer-Logik sowie durch systematisches Testen in kleinen, kontrollierten Schritten. Auch externe Unterstützung, insbesondere durch KI-Tools, half dabei, Denkfehler zu erkennen und Lösungsansätze schneller zu evaluieren.

Insgesamt war dieser Prozess zwar zeitintensiv, hat uns jedoch ein deutlich tieferes Verständnis für Echtzeit-Audioverarbeitung, WebRTC und komplexe Webanwendungen vermittelt, als es eine rein theoretische Auseinandersetzung ermöglicht hätte.

12. Welche Hard- und Software hast du neu kennengelernt, und wie waren deine Erfahrungen?

Durch die Kursarbeit haben wir die Web Audio API sowie die Frameworks p5.js und p5.sound kennengelernt. Die Erfahrungen, die wir dabei sammeln konnten, waren zum Großteil positiv und spannend, auch wenn der Anfang etwas anspruchsvoll war, da Analyse von Audiodaten in Echtzeit für uns komplett neu war.

MK: Durch das Projekt haben wir zahlreiche neue Hard- und Software-Technologien kennengelernt, darunter WebRTC, PeerJS, p5.js im Audio-Kontext sowie eine vertiefte Nutzung der Web Audio API. Insbesondere WebRTC stellte sich als herausfordernd, aber auch als äußerst lehrreich heraus. Die Auseinandersetzung mit dieser Technologie hat unser Verständnis für Echtzeitkommunikation im Web deutlich erweitert.

13. Wie könnte dein Projekt öffentlich zugänglich gemacht werden (Webserver, Plattform, GitHub)?

Für das Projekt bieten sich gut GitHub Pages an, da das Projekt vollständig im Browser läuft. Durch das Bereitstellen des Quellcodes auf GitHub wurde zusätzlich die Möglichkeit für andere Nutzer entstehen das Projekt nicht nur anzusehen, sondern auch mit weiter kreativen Ideen weiterzuentwickeln.